

COMPOSE - PROD STAGE 1

Instrucciones

Continúa sobre el repositorio de [Dev Stage](#) del blog que Laura hizo con Lidia en Backend de segundo de DAW. Esta es la segunda de las tres etapas de dockerización.

Objetivo de esta etapa

Integra únicamente la parte recuadrada en rojo en la primera captura: **Gunicorn como servidor WSGI y la aplicación Django**. Amplía los archivos Docker y Compose de la práctica anterior para sustituir el servidor de desarrollo. Nginx se añadirá en Prod Stage 2.

Trabajo a realizar

1. Añade Gunicorn a las dependencias de producción y úsalo como proceso de arranque del servicio de la aplicación.
2. Localiza el módulo real `personalblog/wsgi.py` y su objeto `application`. El comando de referencia para este proyecto es:

```
gunicorn personalblog.wsgi:application --bind 0.0.0.0:8000
```

0.0.0.0 permite escuchar en las interfaces del contenedor y 8000 es el puerto de la aplicación. Ejecuta el proceso desde el directorio del proyecto, donde está `manage.py`.

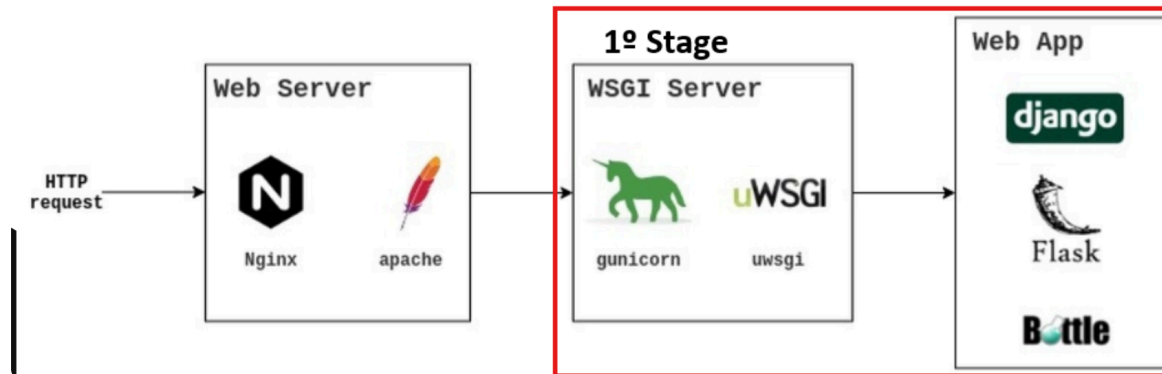
3. Prepara la configuración de producción de Django: desactiva `DEBUG` y configura los `hosts` con los que vas a acceder a la práctica en `ALLOWED_HOSTS`.
4. Retira `django-browser-reload` de las dependencias de producción y de las referencias activas del proyecto. En el ZIP aparece en `INSTALLED_APPS` y `MIDDLEWARE`, dentro de `personalblog/settings.py`, y en la ruta `__reload__` de `personalblog/urls.py`.
5. Comprueba que el contenedor arranca con Gunicorn y que las peticiones llegan a Django. Documenta los cambios respecto a Dev Stage y conserva el trabajo del blog.

En esta etapa se comprueba el funcionamiento de Gunicorn y Django. Con esta configuración, Gunicorn no proporciona el servicio de estáticos que ofrecía `runserver`: la integración de CSS, Tailwind e imágenes mediante Nginx se completa en la siguiente tarea.

Compose – prod deploy 1ºstage

Usamos gunicorn → `gunicorn modulo.archivo-config-wsgi:application --bind 0.0.0.0:8000`

- `wsgi` → creado automáticamente por django
- `0.0.0.0` → contenedor escucha en todas las interfaces
- `:8000` → sirve en el puerto 8000



Integra la parte marcada en rojo: Gunicorn y Django. [Ver diapositiva 136 ↗](#)

Compose – prod deploy 1ºstage

- Problemas → `django-browser-reload`: Recarga dinámica de cambios en dev → mala praxis en prod
 - Envía eventos cada vez que detecta cambios en los archivos para recargarlos
 - Ineficiente

Deberemos eliminar todo rastro de reload: → `requirements.txt` && `/modulo_wsgi` (todas las clases)

`/personalblog`

`django_browser_reload`

Retirada de la recarga de desarrollo en la configuración de producción. [Ver diapositiva 137 ↗](#)

Entrega

Adjunta el enlace al mismo repositorio, actualizado, y amplía el `README.md` con el comando de Gunicorn, los archivos modificados, la retirada de la recarga y capturas de los contenedores, los registros y la aplicación respondiendo. Explica qué queda pendiente para [Prod Stage 2](#).

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



[laura-personal_blog-main.zip](#)

Proyecto original de referencia · continúa sobre tu entrega de Dev Stage · ZIP, 2,6 MB

